

Fall Semester 2025 - 2026

CASE STUDY - I

Course Name :- Data Structure And Algorithm

Course Code :- BCSE2022

Class Number :- VL2025260101553

Faculty Name :- Dr. S.M Prabin

Que-1 Solve the following Recurrence using Master theorem :-

① $T(n) = 3T(\frac{n}{2}) + \sqrt{n}$

Solve $T(n) = aT(n/b) + f(n)$

$a \geq 1$

$b > 1$

$f(n) = \Theta(n^k \log^p n)$

given :- $a=3, b=2, k=0.5, p=0$

Such that $\log_b a \Rightarrow \log_2 3 = 1.585$

If $(\log_b a > k) \rightarrow \text{condition}$

then $\Theta(n^{\log_b a}) \rightarrow \{\text{Case - 1}\}$

for $(1.585 > 0.5)$

then $T(n) = \Theta(n^{\log_2 3})$ Ans

$$\textcircled{2} \quad T(n) = 4T\left(\frac{n}{4}\right) + n^2 \log n$$

given :- $a=4, b=4, k=2, p=1$

To check :- $\log_b a \Rightarrow \log_4 4 = 1$

$$\{1 < 2\} \rightarrow (\text{case - 3})$$

If $(p \geq 0)$ then $\Theta = (n^k \log^p n)$

$$T(n) = \Theta(n^2 \log n) \text{ Ans}$$

$$\textcircled{3} \quad T(n) = 2T\left(\frac{n}{4}\right) + \frac{1}{n}$$

$\Rightarrow a=2, b=4, k=-1, p=0$

To check :- $\log_b a \Rightarrow \log_4 2 = 0.5$

$$(\log_b a > k) \rightarrow \{\text{case - 1}\}$$

$$(0.5 > -1)$$

then $\Theta = (n^k \log^p n)$

$$T(n) = \Theta(n^{1/2})$$

(OR)

$$T(n) = \Theta(\sqrt{n}) \text{ Ans}$$

$$(4) T(n) = 16T\left(\frac{n}{4}\right) - n^2 \log n$$

$\Rightarrow$ given :- $a=16$, $b=4$, $f(n) = -n^2 \log n$

for standard Master theorem

$f(n) \Rightarrow 0$ (non-negative)

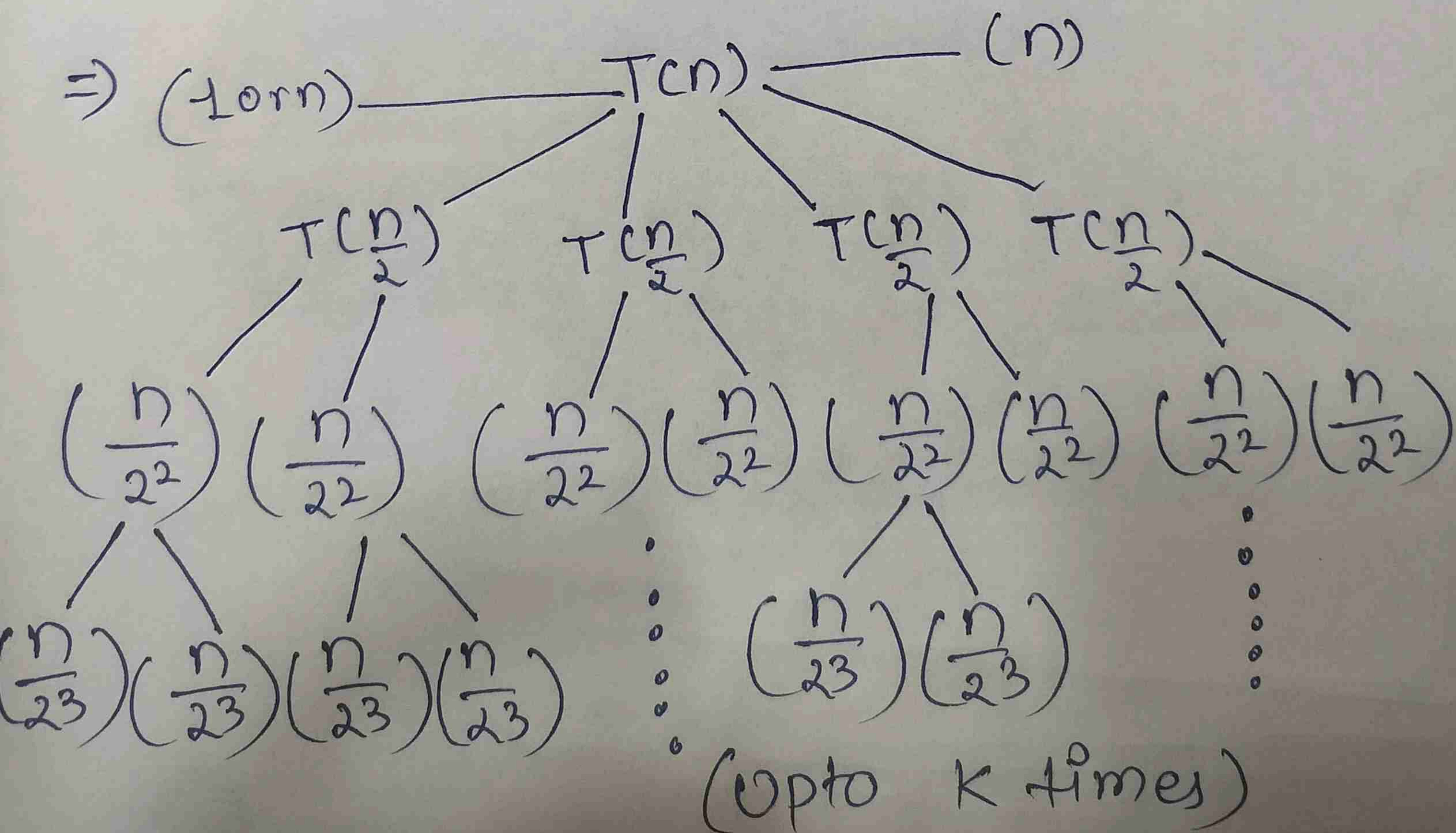
Since $(f(n) = -n^2 \log n < 0)$

So master theorem Does not Apply.

Hence the Answer for the question is not possible.

Que-2 (i) find the time complexity
By recurrence relation of
recurrence tree method

$$(A) T(n) = 4T\left(\frac{n}{2}\right) + n$$



Since $\left\{ \frac{n}{2^k} = 1 \right\} \rightarrow$ Assumed

$$n = 2^k \Rightarrow k = \log_2 n$$

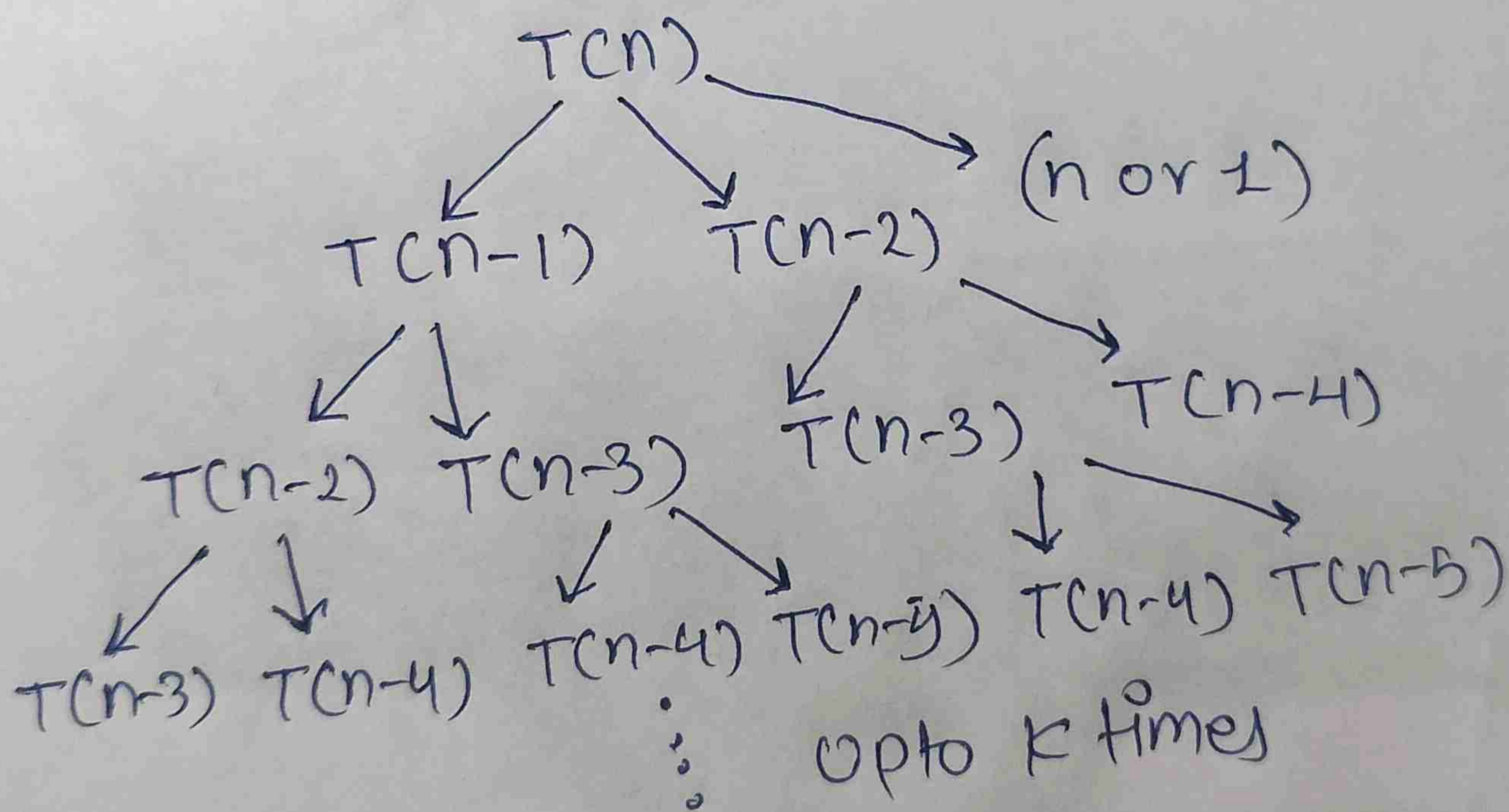
The number of leaves $\Rightarrow 4^k \Rightarrow 4^{\log_2 n}$

$$(2^2)^{\log_2 n} \Rightarrow 2^2 \log_2 n$$

$$\Rightarrow (2 \log_2 n)^2 \Rightarrow n^2$$

Then $T(n) = \Theta(n^2)$ Ans

$$(b) T(n) = T(n-2) + T(n-1) + n$$



$f(n) = f(n-1) + f(n-2)$ which defines fibonacci series numbers.

time complexity $\Rightarrow \Theta(\phi^n)$ or $\phi = 1.618$ Ans

Ques-2 (ii) Solve the following recurrence using Substitution Method :-

$$\textcircled{1} T(n) = \begin{cases} 1, & \text{if } n=1 \\ T(n-1) + n(n-1), & \text{if } n \geq 2 \end{cases}$$

$$\Rightarrow T(n) = T(n-1) + n(n-1)$$

Substitute $T(n-1)$ by the recurrence

$$\begin{aligned} T(n) &= [T(n-2) + (n-1)(n-2)] + n(n-1) \\ &= T(n-2) + (n-1)(n-2) + n(n-1) \end{aligned}$$

Substitute $T(n-2)$

$$\begin{aligned} T(n) &= [T(n-3) + (n-2)(n-3)] + (n-1)(n-2) + n(n-1) \\ &= T(n-3) + (n-2)(n-3) + (n-1)(n-2) + n(n-1) \end{aligned}$$

After j substitutions (unrolling j times)
we get :-

$$T(n) = T(n-j) + \sum_{k=0}^{j-1} (n-k)(n-k-1),$$

Because at step k ($k=0$) is $(n-k)(n-k-1)$

Stop when $T(n-j)$ reaches the base case.

Take $j = n-1$ so that $T(n-j) = T(1)$

$$T(n) = T(1) + \sum_{k=0}^{n-2} (n-k)(n-k-1)$$

Change Index : let $i = n - k$, when $k = 0 \rightarrow i = n$
when $k = n - 2 \rightarrow i = 2$

So the Sum Becomes

$$\sum_{i=2}^n i(i-1) \quad \left(\begin{array}{l} \text{we can include } i=1 \\ \text{Because } 1 \cdot 0 = 0 \end{array} \right)$$

$$T(n) = T(1) + \sum_{i=1}^n i(i-1)$$

use the standard formula $\sum_{i=1}^n i = \frac{n(n+1)}{2}$

$$= \sum_{i=1}^n (i^2 - i) = \left(\sum_{i=1}^n i^2 \right) - \left(\sum_{i=1}^n i \right)$$

$$= \sum_{i=1}^n i^2 = \frac{n(n+1)(2n+1)}{6}$$

$$\text{So } \sum_{i=1}^n i(i-1) = \frac{n(n+1)}{2} \Rightarrow \frac{n(n+1)(2n+1)}{6} - \frac{n(n+1)}{6}$$

So closed form Substitution is :-

$$T(n) = T(1) + \frac{n(n+1)(n-1)}{3}$$

which implies to $T(n) = \frac{T(1) + n^3 - n}{3}$

Time complexity $\Rightarrow T(n) = \Theta(n^3)$ Ans

$$\textcircled{2} \quad T(n) = 2T(\sqrt{n}) + 1$$

$\Rightarrow$ Change of variable

put $m = \log_2 n$. Then $\sqrt{n} = 2^{m/2}$

So if we define,

$$S(m) = T(2^m)$$

the recurrence becomes,

$$T(n) = 2T(\sqrt{n}) + 1 \iff S(m) = 2S\left(\frac{m}{2}\right) + 1$$

unroll $S(m) = 2S\left(\frac{m}{2}\right) + 1$

$$S(m) = 2S\left(\frac{m}{2}\right) + 1$$

$$= 2\left(2S\left(\frac{m}{4}\right) + 1\right) + 1 \Rightarrow 4S\left(\frac{m}{4}\right) + 3$$

$$= 4\left(2S\left(\frac{m}{8}\right) + 1\right) + 3 \Rightarrow 8S\left(\frac{m}{8}\right) + 7$$

$\vdots$

$$= 2^i S\left(\frac{m}{2^i}\right) + (2^i - 1)$$

Stop when $\frac{m}{2^i} = 1$

such that $i = \log_2 m$ Then,

$$S(m) = 2^{\log_2 m} S(1) + (2^{\log_2 m} - 1) \\ = m \cdot S(1) + (m - 1)$$

$$S(m) = m(S(1) + 1) - 1$$

Here $S(1) = T(2^1) = T(2)$

• Convert back to n , Substitute,

$$T(n) = S(\log_2 n) = (\log_2 n)(S(1) + 1) - 1$$

write constant terms of $T(2)$:-

$$T(n) = (\log_2 n)(T(2) + 1) - 1$$

if your Base is $T(1)$ instead

$$T(n) = (\log_2 n)(T(1) + 1) - 1$$

Time complexity $\Rightarrow T(n) = \Theta(\log n)$

Ans

Ques-3 Illustrate the conversion of the following Infix Expression $(b - c * a + (d * e - f) / g * (f^2))$ to postfix and Prefix Expression and give the Algorithm.

Solve Infix to postfix :-

Input	Stack	Output (Postfix so far)
b	empty	b
-	-	b
c	-	bc
*	- *	bc
a	- *	bc a
+	+	bc a * -
(	+ (	bc a * -
d	+ (	bc a * - d
*	+ (*	bc a * - d
e	+ (*	bc a * - d e
-	+ (-	bc a * - d e *
f	+ (-	bc a * - d e * f

Input	Stack	Output (postfix so far)
)	+	bca*-de*f-
/	+ /	bca*-de*f-
g	+ /	bca*-de*f-g
*	+ *	bca*-de*f-g /
(	+ *(	bca*-de*f-g /
f	+ *(	bca*-de*f-g / f
^	+ *(^	bca*-de*f-g / f
2	+ *(^	bca*-de*f-g / f 2
)	+ *	bca*-de*f-g / f 2 ^

Now pop out Stack operator

{ bca*-de*f-g / f 2 ^ * + }

⇓

{ Infix to postfix
Expression }

⇒ To get prefix Expression we can get

⇒ Postfix → prefix (Reverse Postfix)

Input	Action	Stack (Bottom → top)
b	push	b
c	push	b c
a	push	b c a
*	pop a, c → *ca	b *ca
-	-b *ca	-b *ca
d	push	-b *ca d
e	push	-b *ca d e
*	*de (push)	-b *ca *de
f	push	-b *ca *de f
-	pop f	-b *ca -de f g
g	push	-b *ca -de f g
/	pop g	-b *ca / - *de f g
+	push	-b *ca / - *de f g +
2	push	-b *ca / - *de f g + 2
^	pop 2	-b *ca / - *de f g ^ f 2
*	pop ^ f 2	-b *ca * / - *de f g ^ f 2
+	pop	

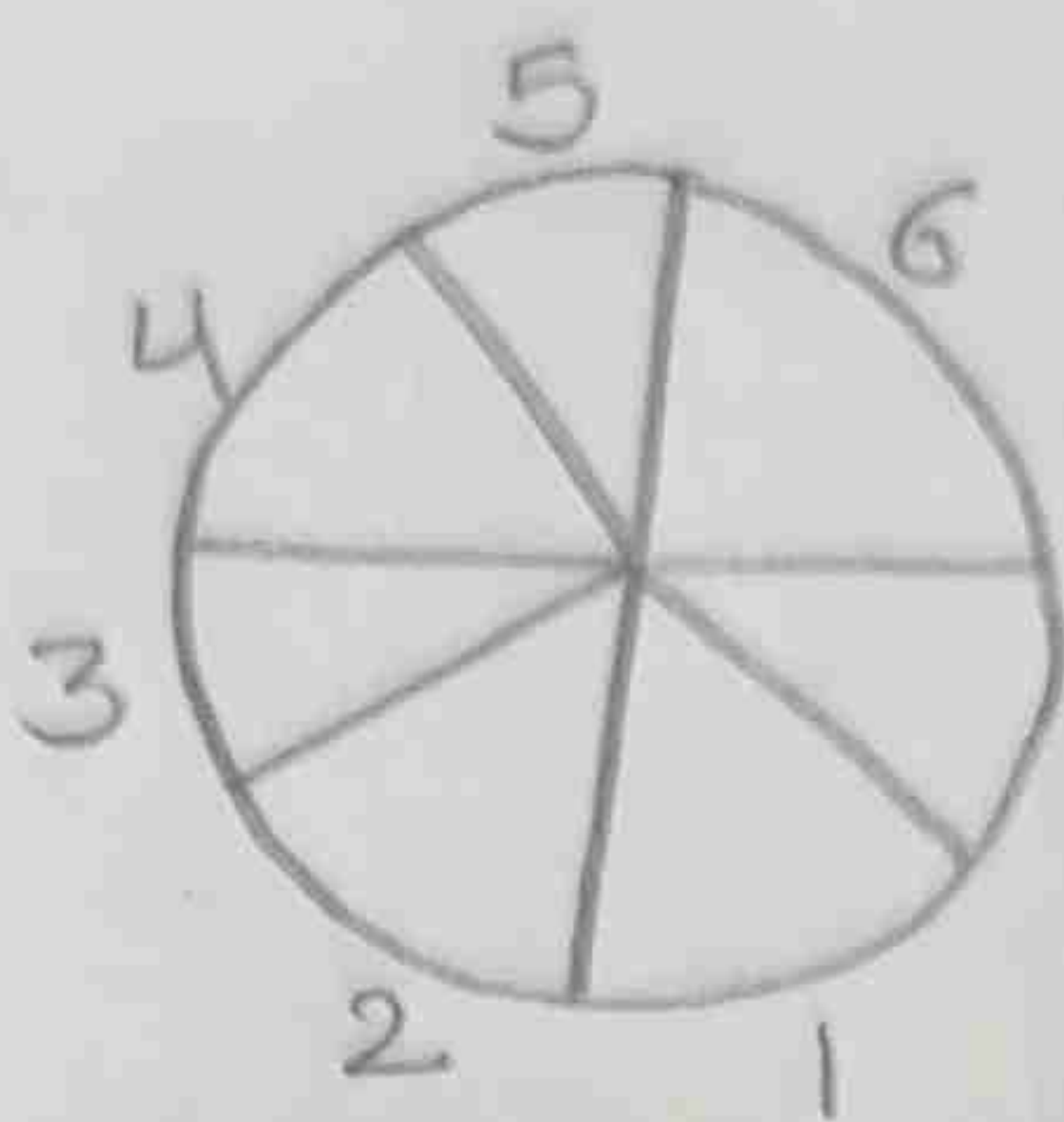
$$\left. \begin{array}{l} +b *ca * / - \\ * / - *de f \\ g \wedge f 2 \end{array} \right\}$$

$$\Downarrow$$
prefix Expression

Ques-4 Solve the following Question By Circular Queue And Draw Diagram.

Circular Queue = null

Command = null



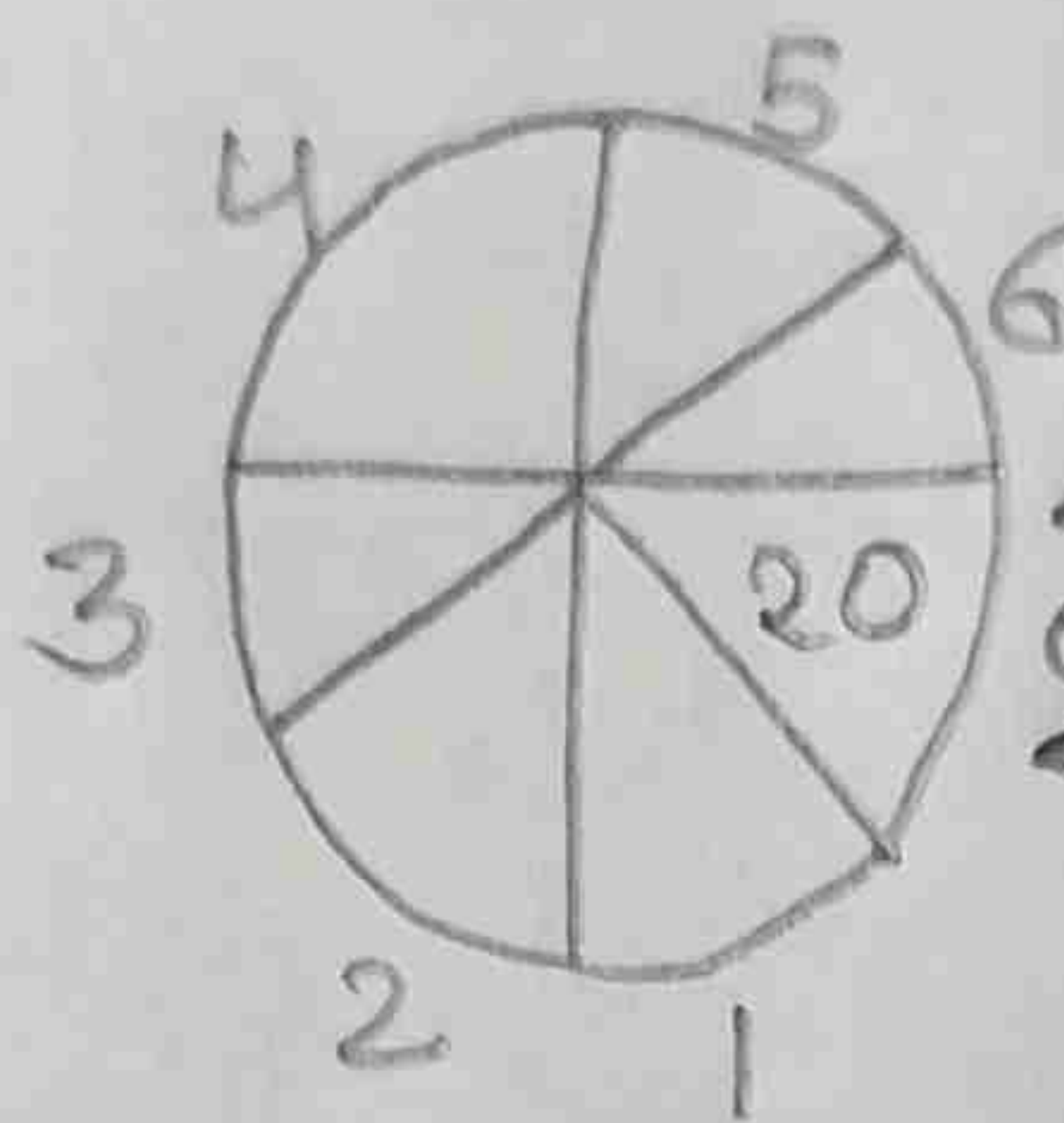
front = -1

Rear = -1

(Empty Queue)

Circular Queue = 20

Command Enqueue (20)



front

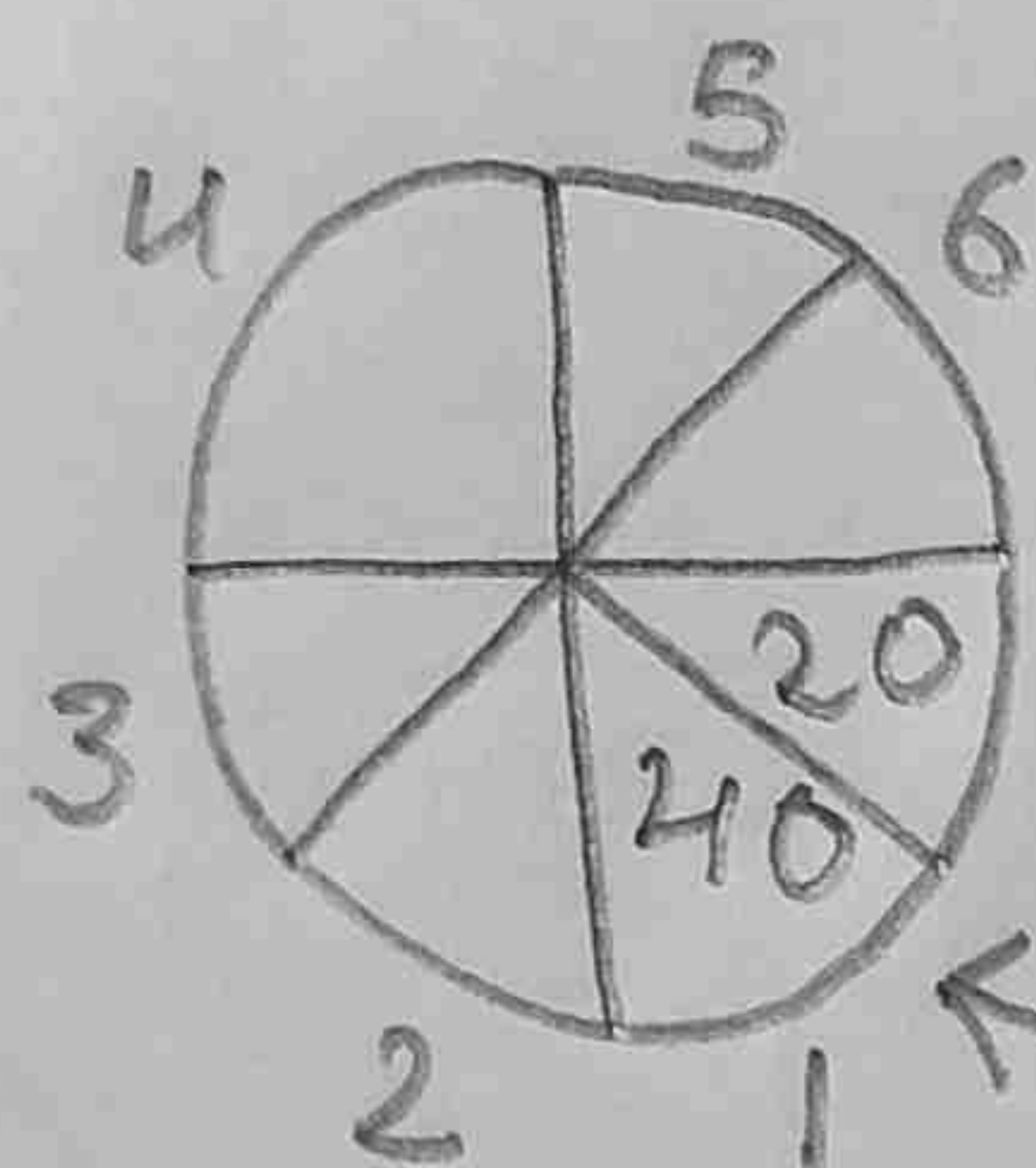
front = 0

Rear

Rear = 0 (20)

Circular Queue = 20, 40

Command Enqueue (40)



front

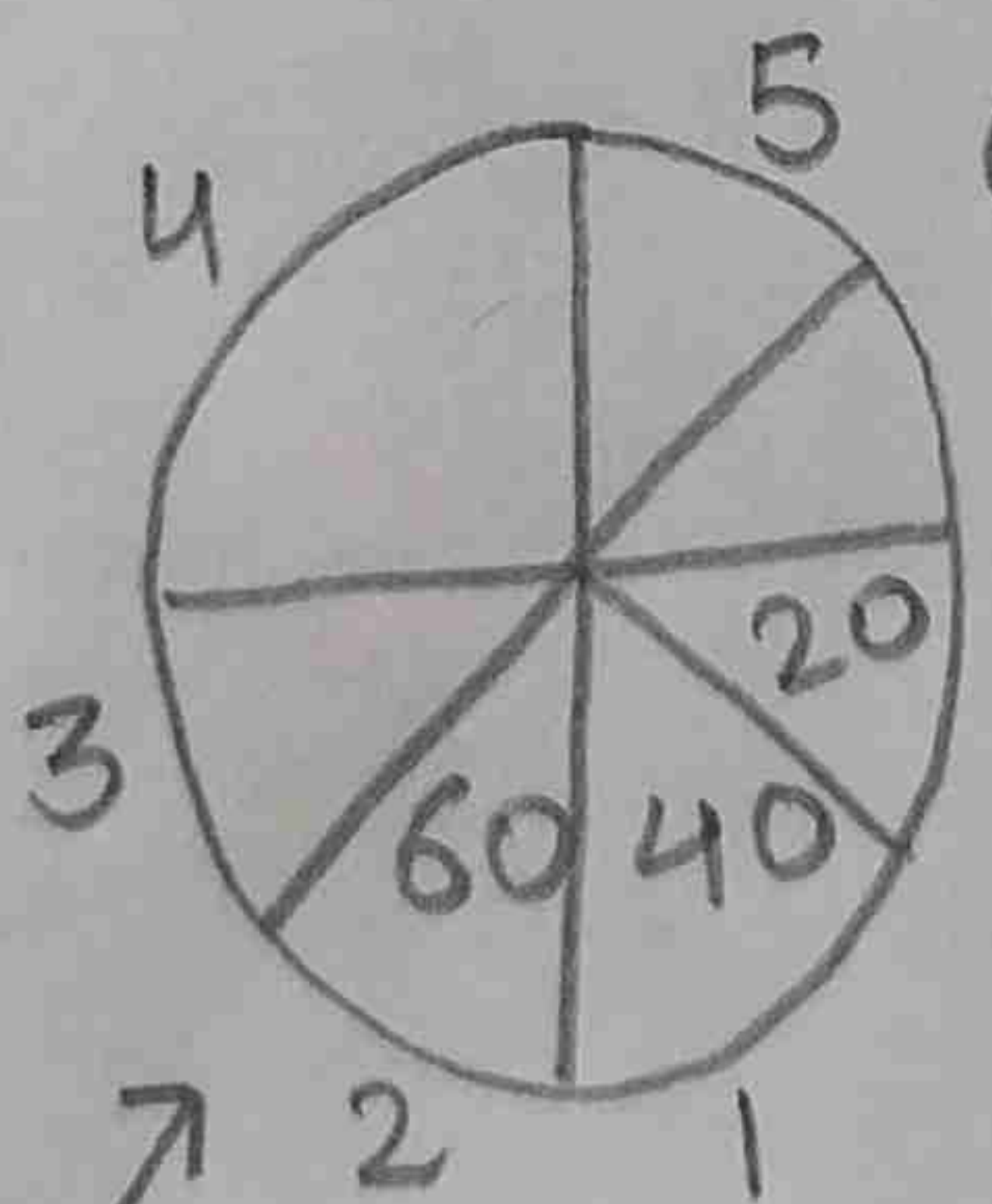
front = 0

Rear

Rear = 1 (40)

Circular Queue = 20, 40, 60

Command Enqueue (60)



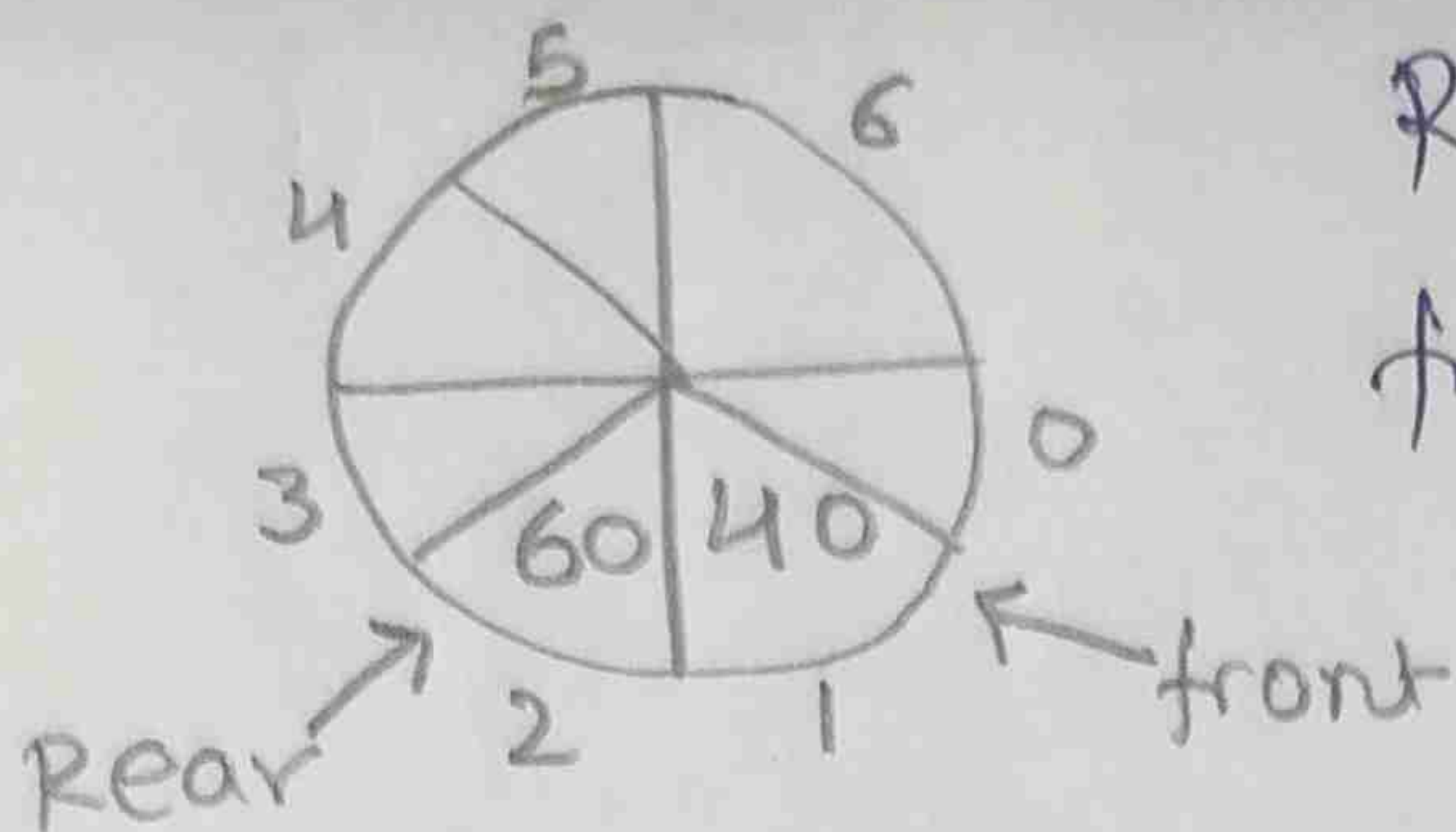
front

front = 0

Rear = 2 (60)

Circular Queue = 40, 60

Command Dequeue (20)

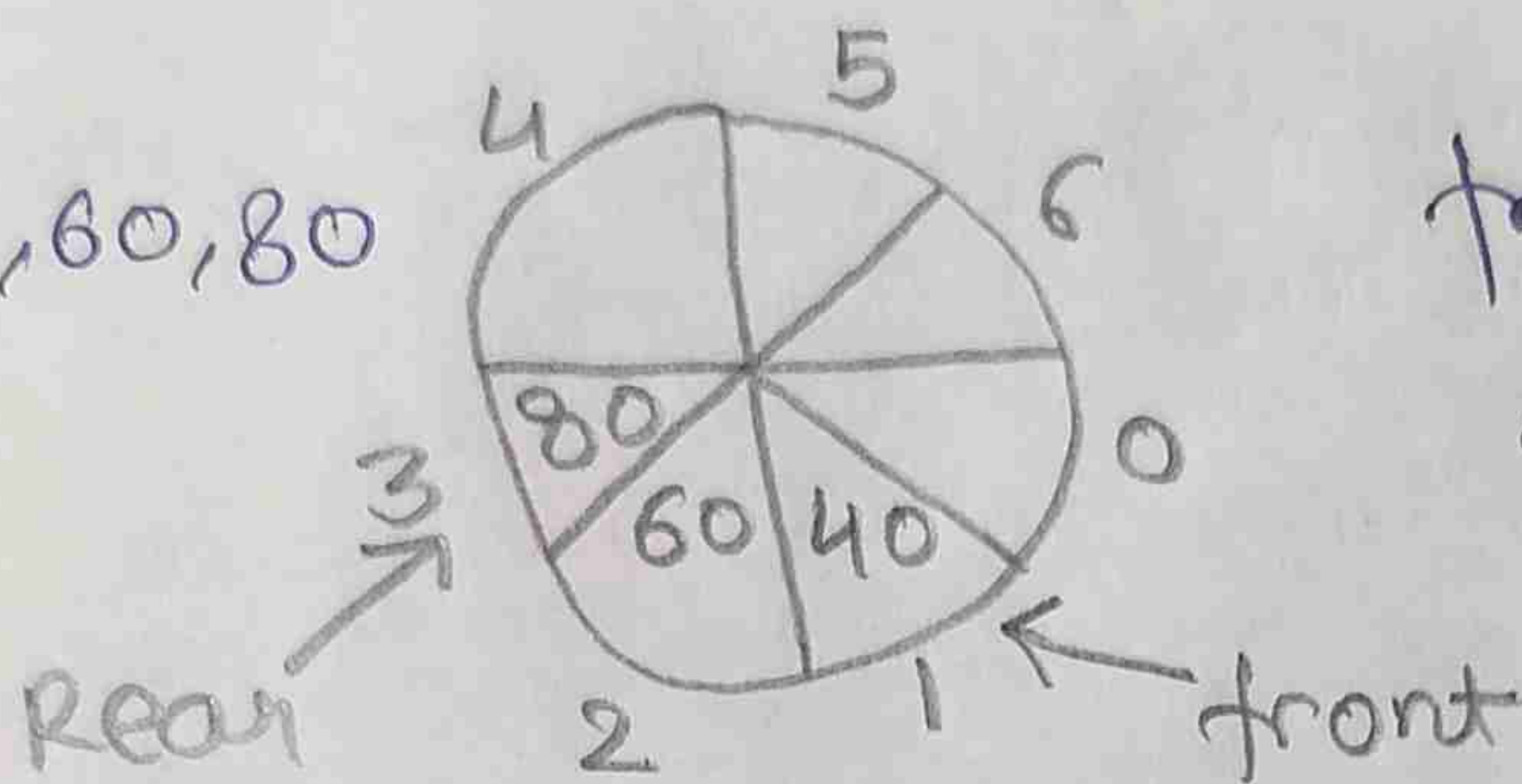


Rear = 2 (60)

front = 1 (40)

Circular Queue = 40, 60, 80

Command Enqueue (80)

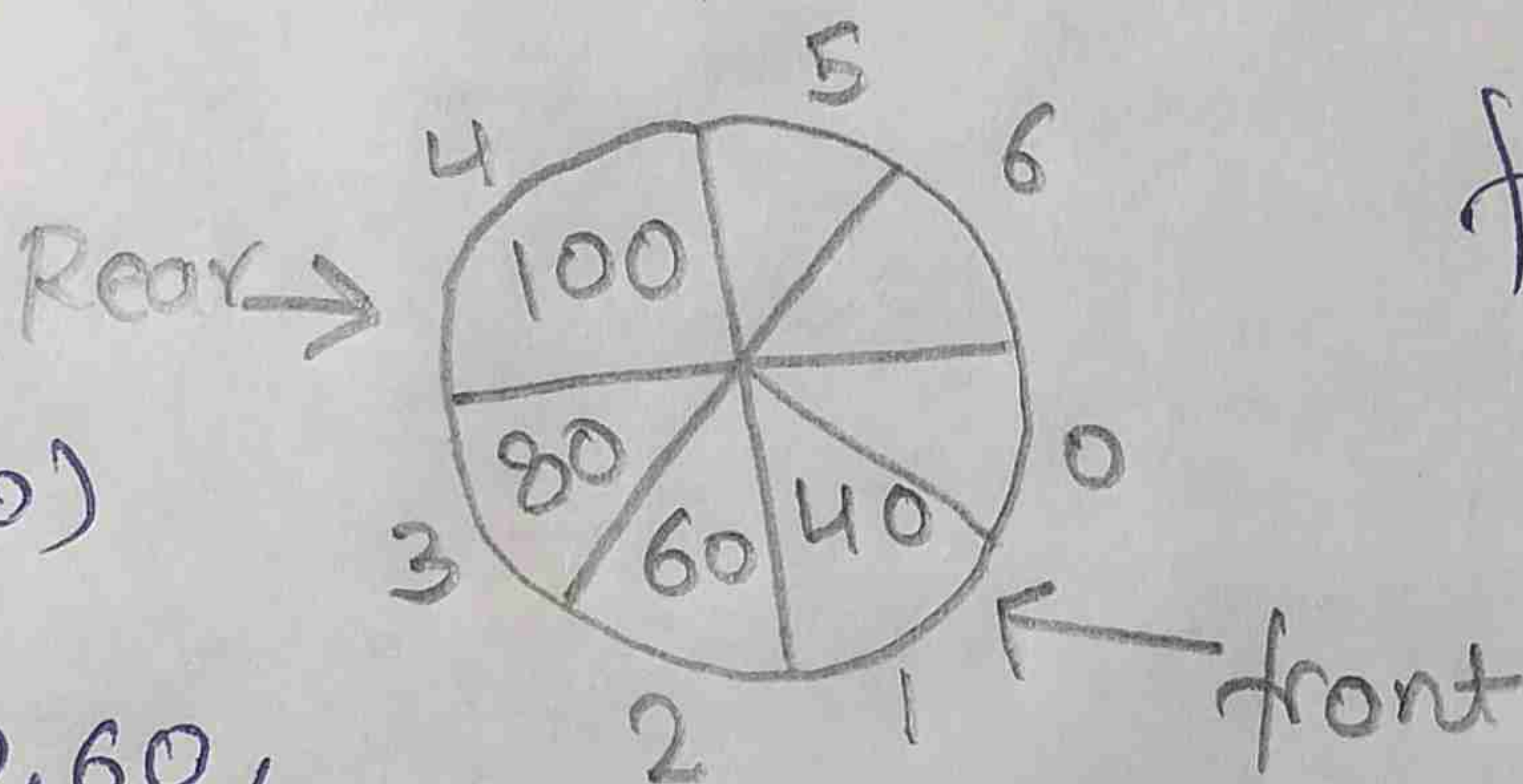


front = 1 (40)

Rear = 3 (80)

Command Enqueue (100)

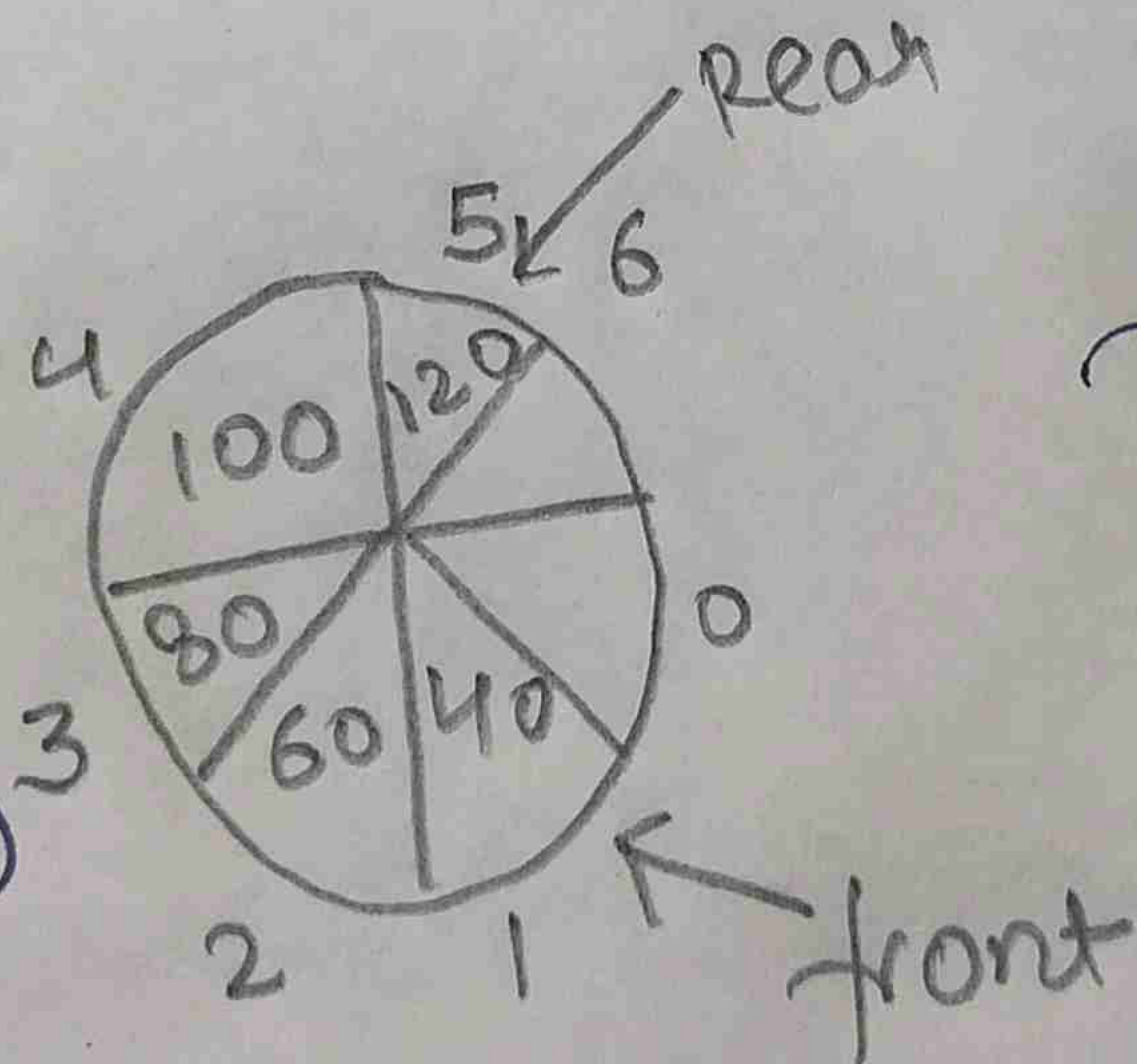
Circular Queue = 40, 60, 80, 100



front = 1 (40)

Rear = 4 (100)

Circular Queue (40, 60, 80, 100, 120)



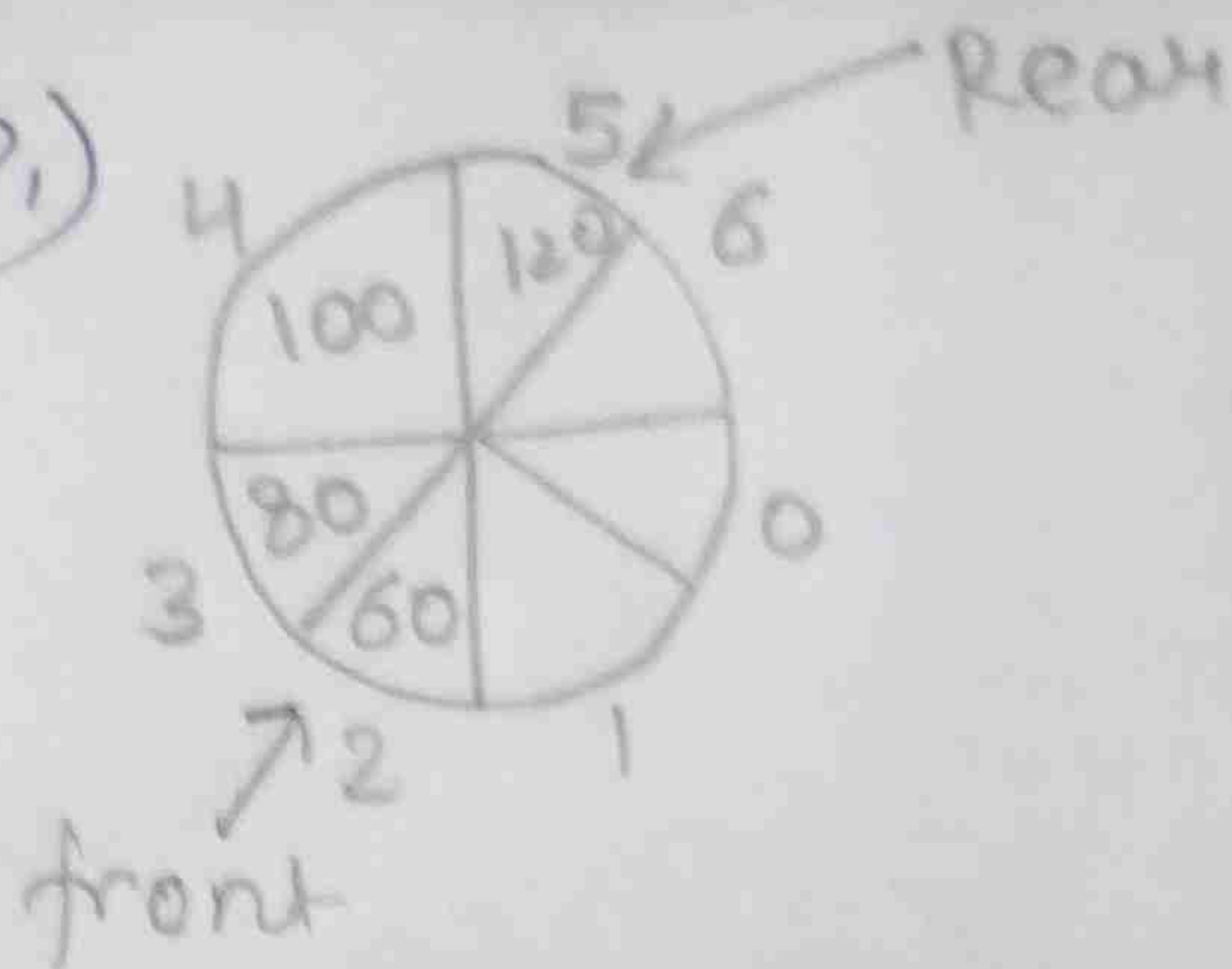
front = 1 (40)

Rear = 5 (120)

Command Enqueue (120)

Circular Queue (60, 80, 100, 120)

Command Dequeue (40)

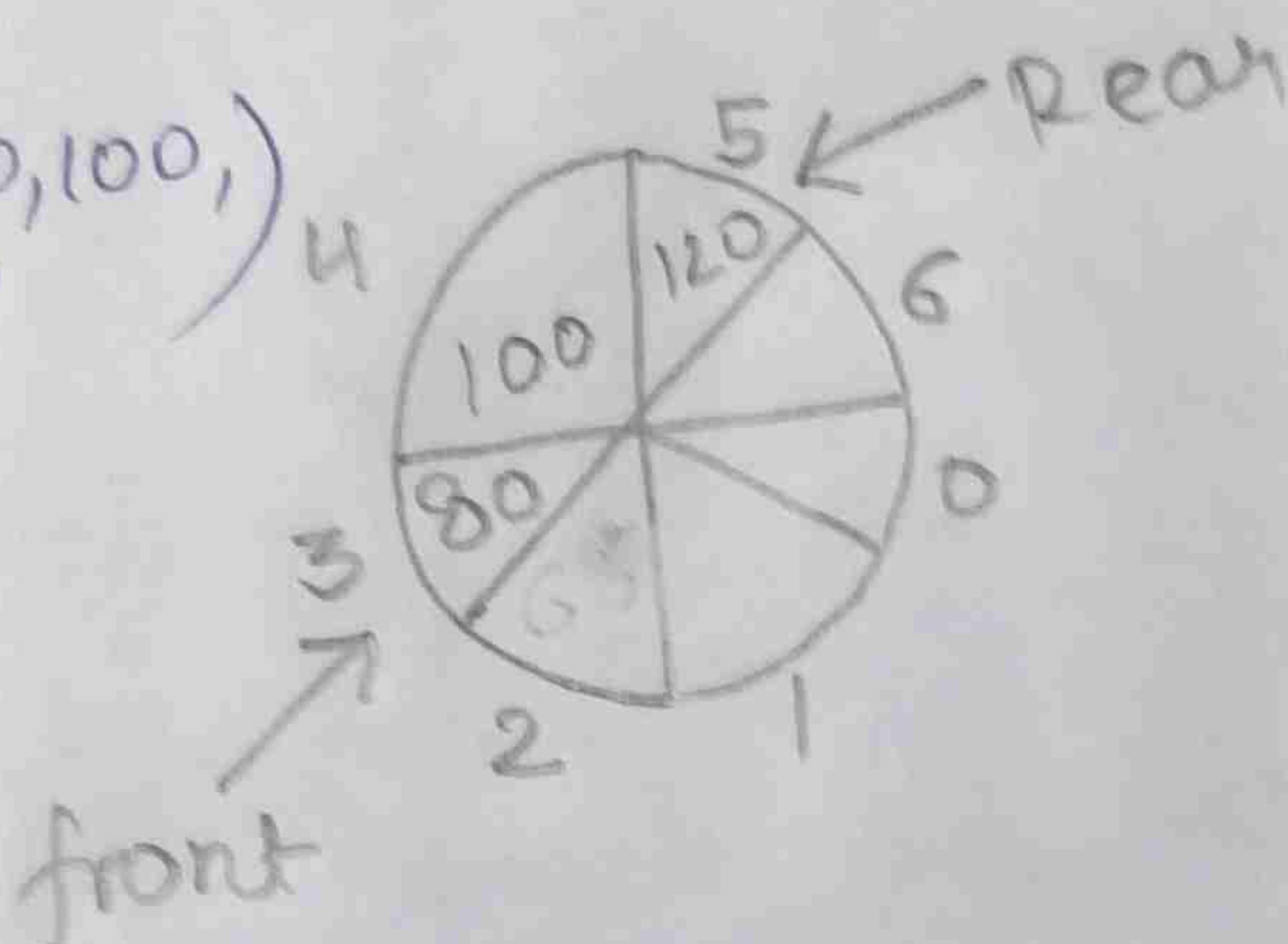


front = 2 (60)

Rear = 5 (120)

Circular Queue (60, 80, 100, 120)

Command Dequeue (60)

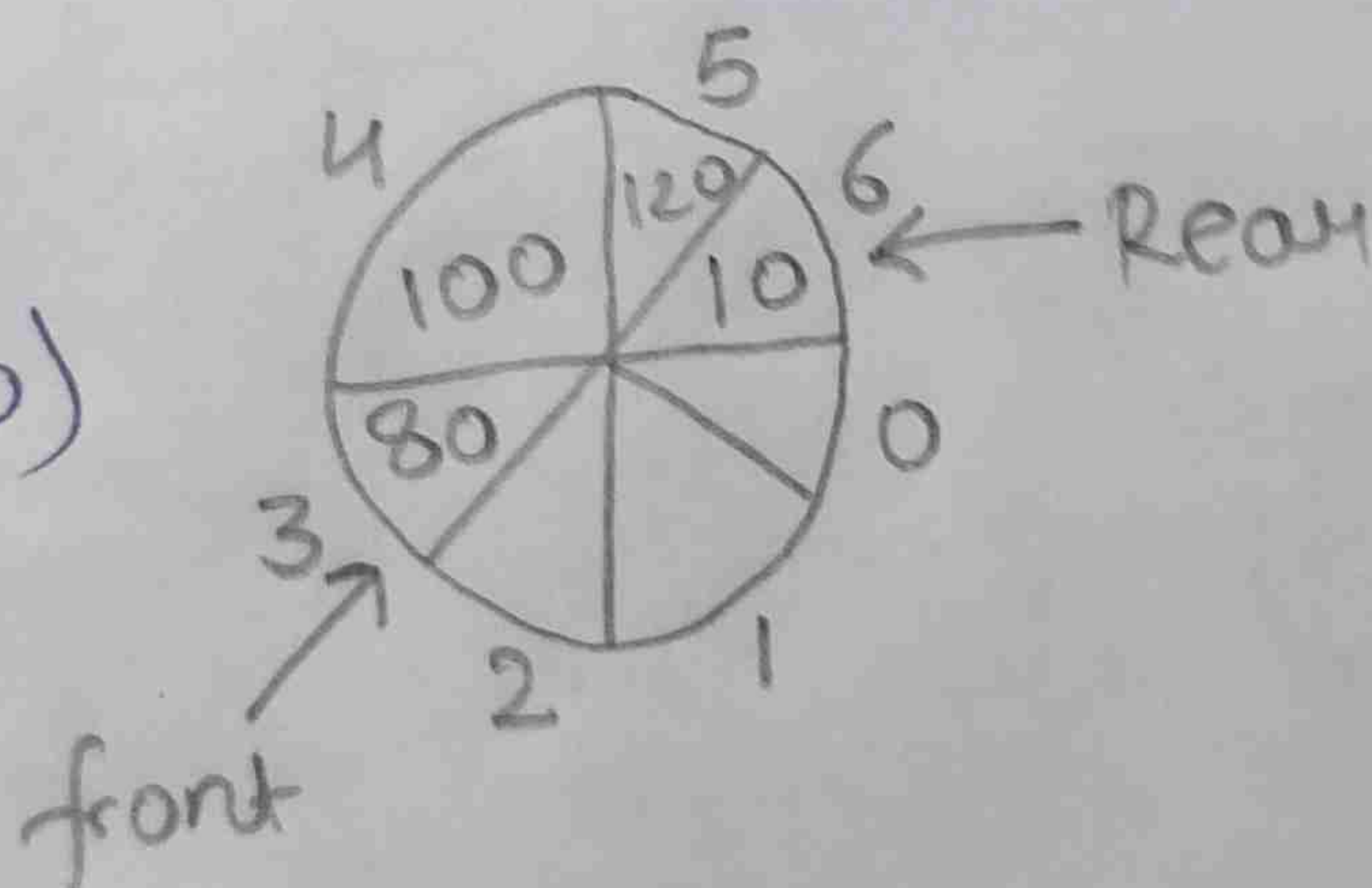


front = 3 (80)

Rear = 5 (120)

Circular Queue (80, 100, 120, 10)

Command Enqueue (10)



front = 3 (80)

Rear = 6 (10)

⇓
(final positions)

Ques-5 Differentiate iterative procedure And Recursive procedure. Write the recursive algorithm for finding the power of a given number and trace the algorithm for power values of (0, 1, odd, even). Derive the Recurrence relation for the power function and Analyze its time complexity.

Ans (Iterative)

- ① uses loops (for/while) to Repeat steps.
- ② state kept in variables, no call stack overhead.
- ③ Typically constant extra space:
Space = $O(1)$.
- ④ Example (naive power): multiply (x) by itself (n) times.

(Recursive)

- ① function calls itself with a smaller subproblem.
- ② state kept via call stack, has overhead per call.
- ③ Space Depends on Recursion Depth.
Ex - $O(\log n)$.
- ④ often cleaner / shorter logic for Divide and Conquer problems.

② Recursive Power Algorithm :- (Exponential) By Sawaring

• Pseudocode :-

Power (x, n)

- ① if $n == 0$; return 1
- ② if $n == 1$; return x
- ③ if n is even :
- ④ $y = \text{power}(x, n/2)$
- ⑤ return $y * y$
- ⑥ else (n is odd):
- ⑦ $y = \text{power}(x, (n-1)/2)$
- ⑧ return $x * y * y$

⇒ Explanation of steps :-

- Line 1-2 (Best Cases)
if $n = 0 \rightarrow$ return 1 (Definition of power)
if $n = 1 \rightarrow$ return x
- Line 3-5 (Even Cases)
for even n, $x^n = (x^{(n/2)})^2$.
so compute once, save it.
- Line 6-8 (Odd Cases)
for odd n, $x^n = x * (x^{((n-1)/2)})^2$
multiply extra(x) to account for odd power.

③ Tracing the Algorithm :-

- for case $n=0$, $n=1$, even, odd

① case-1 : ($n=0$)

$$\text{POWER}(x, 0) \rightarrow \text{return } 1 \quad \{x^0 = 1\}$$

② case-2 : ($n=1$)

$$\text{POWER}(x, 1) \rightarrow \text{return } x \quad \{x^1 = x\}$$

③ case-3 : n is even (Ex: $x=3$, $n=8$)

$$\begin{aligned} &\text{POWER}(3, 8) \\ &= (\text{POWER}(3, 4))^2 \\ &= ((\text{POWER}(3, 2))^2)^2 \\ &= (((\text{POWER}(3, 1))^2)^2)^2 \\ &= (((3^1)^2)^2)^2 \\ &= (9^2)^2 \\ &= 81^2 \\ &= (6561) \end{aligned}$$

$$\text{Hence } \{3^8 = 6561\}$$

④ case-4 : n is odd (Ex: $x=2$, $n=5$)

$$\begin{aligned} &\text{POWER}(2, 5) \\ &= 2 * (\text{POWER}(2, 2))^2 \\ &= 2 * ((\text{POWER}(2, 1))^2)^2 \\ &= 2 * (2^2)^2 \end{aligned}$$

$$= 2 * 4^2$$

$$= 2 * 16$$

$$= 32$$

$$\text{Hence } \{2^5 = 32\}$$

④ Recurrence Relation :-

• Let $T(n)$ = time taken to compute $\text{POWER}(x, n)$

• Base : $T(0) = O(1)$, $T(1) = O(1)$

• for $(n \geq 2)$

① If n is even : $T(n) = T(n/2) + O(1)$

② If n is odd : $T(n) = T((n-1)/2) + O(1)$

Both are essential $T(n) = T(\lfloor n/2 \rfloor) + O(1)$

⑤ Solving the Recurrence :-

$$T(n) = T(n/2) + O(1)$$

$$= T(n/4) + O(1) + O(1)$$

$$= T(n/8) + 3O(1)$$

$$\vdots$$

$$= T(1) + \log_2(n) * O(1)$$

Thus,

$$T(n) = O(\log n)$$

⑤ correctness proof :-

① Base case :-

for $n=0$, $\text{POWER}(x, 0) = 1$

for $n=1$, $\text{POWER}(x, 1) = x$

Both true

② Inductive steps :-

Assume correct for all $(k < n)$

• If n is even :

By Induction $\Rightarrow \text{result} = x^n$

• If n is odd :

By Induction $\Rightarrow \text{result} = x^n$

Hence proved By Induction.

⑥ Space Complexity :-

① Each Recursive call reduces (n) by 1.
Halt.

② Maximum recursion depth $= \log_2(n)$

Space complexity $\Rightarrow O(\log n)$ { Due to call stack }